Chapter 8: Creating ADO Database Applications

ActiveX Data Objects (ADO) acts as the bridge between OLE DB and your applications. OLE DB has a programming interface that is designed for optimal functionality in a wide variety of applications, but it does not meet the requirement for a simple programming interface. ADO is an API that simplifies OLE DB.

The ADO programming model defines the sequence of activities necessary to gain access to and update a data source. The model is based on a set of objects where each object simplifies a specific database operation, such as connecting to a database or retrieving data. Objects possess methods and properties. Methods perform some operation on data. Properties either represent some attribute of the data, or control the behavior of some object method. Associated with objects are events, which are notifications that some operation has occurred, or is about to occur.

In this chapter, you will learn to use the ADO programming model to create applications that use data controls and data-bound dialog boxes to access databases. You will also learn to perform queries and find records in a database table.

Objectives

After completing this chapter, you will be able to:

- Describe the role of ADO in applications that require database access.
- Describe the ADO programming model.
- Implement ADO in MFC-based applications by using data controls.
- Implement ADO in MFC-based applications by using the data-bound dialog box.
- Perform queries and searches on databases and rowsets.

247 Introduction to ADO

ADO helps you develop applications that access and manipulate data through an OLE DB provider. ADO provides a set of objects that simplify database-related operations, such as connecting to a database and retrieving rowsets.

This section begins with an introduction to ADO. You will learn about the features of ADO and benefits of using ADO to access data. You will also look at the ADO object model and see what objects, properties, and methods are available.

This section also introduces you to the two approaches to implementing ADO technology in MFC-based applications.

This section includes the following topics:

- Benefits of Using ADO
- ADO Object Model
- Using ADO in MFC-based Applications

Notes

248 Benefits of Using ADO

ADO is designed as an easy-to-use application-level interface to OLE DB. OLE DB provides high-performance access to any data source. A data source includes relational and nonrelational databases, e-mail, and file systems. ADO provides a minimal number of layers between your application and data source to provide a lightweight, high-performance interface.

ADO is easy to use because it is based on the generic Automation interface, which is supported by several tools and languages. ADO combines the best features of Remote Data Objects (RDO) and Data Access Objects (DAO), using similar conventions with simplified semantics that make it easy to learn.

In addition, ADO provides high speed and low memory overhead for improved performance.

- De-emphasizes the object hierarchy

  You need not navigate through a hierarchy to create objects, because objects are independent. This model results in fewer objects and a smaller working set. You track and create only the relevant objects.
- Supports free-threaded objects for efficient Web server applications
- Supports batch processing and stored procedures with in/out parameters and return values

  ADO also supports retrieval of multiple recordsets by stored procedures and batch statements.
- Provides advanced rowset cache management for better performance

To see an animation that describes the need for ADO and the ADO object model, click this icon.
 (CD-ROM plays the animation, "ADO Object Model.")

Notes

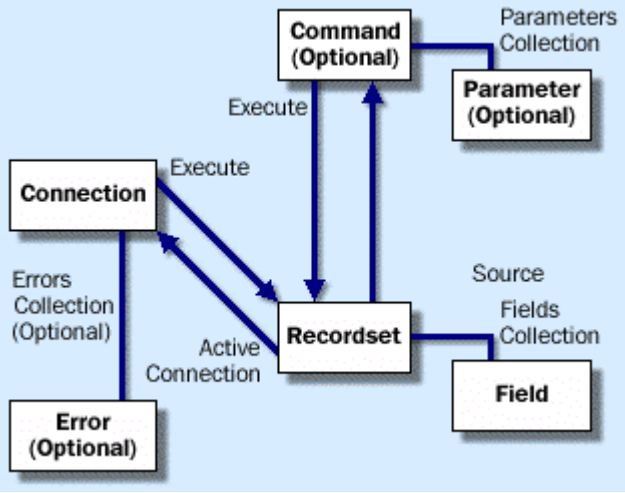
249 ADO Object Model

The ADO 2.0 object model consists of the following:

- Objects and properties
- Collections
- Methods

Objects and Properties

The following illustration shows the ADO objects and how they are related. Click the objects in this illustration to display a description of each object.



(CD-ROM displays context-sensitive text.)

Each ADO object has a set of properties associated with it. You can customize data access operations by modifying the values of these properties.

The properties of an ADO object can be classified into two categories:

- Dynamic properties

  These properties are defined by the underlying data provider. Each such property is represented as a **Property** object.
- Built-in properties

  These are native ADO properties and are automatically available to any newly created object.

For a complete list of ADO object properties, see "ADO Properties" in ADO Documentation, OLE DB SDK 1.5.

Notes

Collections

A collection refers to a group of similar objects owned by another object. For example a **Rowset** object owns a collection of **Field** objects, where each **Field** object represents a column in the rowset.

The ADO object model has the following collections:

- **Errors** collection

  A **Connection** object contains an **Errors** collection, which contains all the **Error** objects created in response to a single failure involving the data provider.

- **Parameters** collection

  A **Command** object contains a **Parameters** collection made up of **Parameter** objects. Each **Parameter** object represents a parameter of the specific command.

- **Properties** collection

  Each ADO object has a **Properties** collection, which contains a set of **Property** objects. Each **Property** object represents a dynamic property of the specific instance of an object.

- **Fields** collection

  A **Recordset** object has a **Fields** collection made up of **Field** objects. Each **Field** object represents a column in the recordset.

Methods

The following table lists some ADO methods that you can use to implement ADO in your applications.

Method	Description	Applies to
MoveFirst, **MoveLast**, **MoveNext**, **MovePrevious**	Moves to the first, last, next, or previous record in a specified **Recordset** object and makes that record the current record.	**Recordset** object
Open	Opens a connection to a data source.	**Connection** object
GetRows	Retrieves multiple records of a recordset into an array.	**Recordset** object

For a complete list of ADO methods, see "ADO Methods" in ADO Documentation, OLE DB SDK 1.5.

Notes

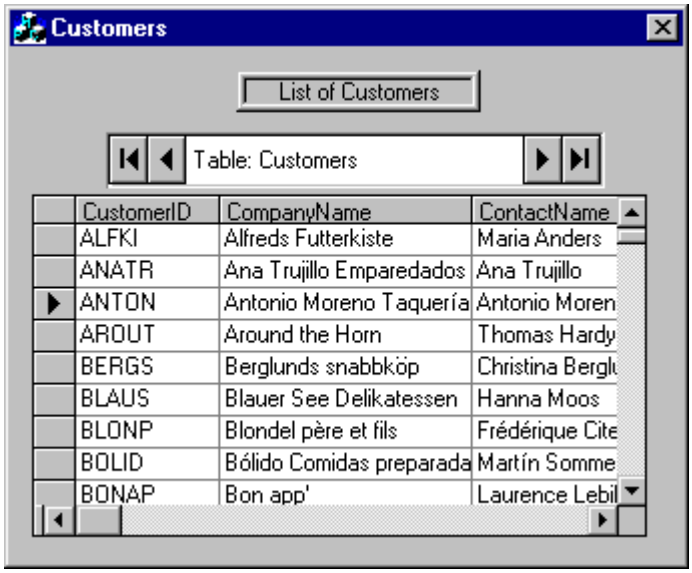
250

Using ADO in MFC-based Applications

You can create an ADO application in two ways: by using data controls and by using the data-bound dialog box.

Using the Data Controls

You can use the data controls to access a data source. The following illustration shows an application that uses this approach to implement ADO.



The ADO data control connects to the database and retrieves recordsets. A data-bound control binds to the data control and displays the records retrieved from a database.

For information about how to implement ADO in an MFC-based application by using this technique, see Implementing ADO Using Data Controls in this chapter.

Using the Data-bound Dialog Box

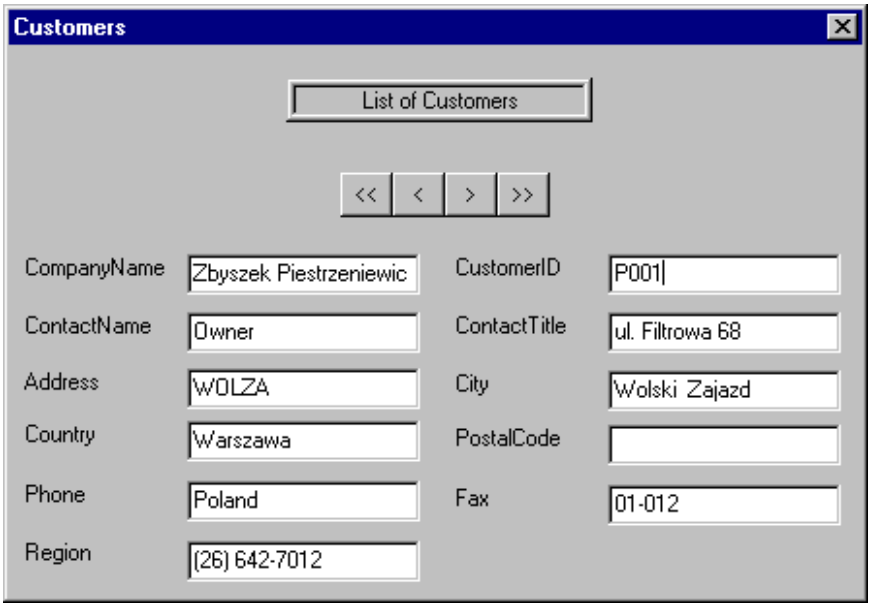
In addition to using data controls, you can create a data-bound dialog box to implement ADO.

Note If you are using the Enterprise Edition of Visual C++, you can use the ADO Data Bound Dialog Wizard to create the dialog box. If you are using the Professional or Standard Edition, you need to create the dialog box manually.

The data-bound dialog box contains controls, such as edit boxes and check boxes, that are bound to the fields of a recordset.

Notes

The following illustration shows an application with a data-bound dialog box.



The data-bound dialog box provides you record-level access to your data. You can perform database operations, such as adding new records and modifying existing records.

For information about how to create a data-bound dialog box, see Using the Data-bound Dialog Box in this chapter.

Notes

251 Using Data Controls

In this section, you will learn to use the data controls—the ADO data control and the data-bound controls—to create an ADO application.

The ADO data control connects to the database and retrieves rowsets. A data-bound control binds to the ADO data control and displays recordsets retrieved from a database.

This section includes the following topics:

- The ADO Data Control
- Data-bound Controls
- Implementing ADO Using Data Controls

Notes

252 The ADO Data Control

The ADO data control is a graphic control that provides an easy-to-use interface to help you develop applications with minimum code. Instead of using ADO objects, such as **Connection** and **Recordset**, you can use the ADO data control to quickly create a connection to a database and retrieve records.

The following illustration shows an ADO data control.



The ADO data control establishes a connection between a data provider and a data-bound control. A data-bound control is any Windows control that has a **DataSource** property. For more information about data-bound controls, see Data-bound Controls in this chapter.

The ADO data control performs the following functions:

- Connects to a local or remote database
- Opens a specified database table or defines a set of records based on an SQL query or stored procedure
- Supplies field values to data-bound controls
- Enables addition or modification of field values displayed in data-bound controls, depending on the functionality supported by the provider and the data source

For a step-by-step procedure of how to use an ADO data control to implement a client application, see Implementing ADO Using Data Controls in this chapter.

Properties of an ADO Data Control

The following table lists some important properties of the ADO data control that you can set to define a connection to a database.

Property	Description
Provider	Specifies the data provider.
DataSourceName	Specifies the name of the data source.
UserName and Password	Specifies the name of the user and password, if the database is password-protected.
RecordSource	Defines the recordset. Could either be a table name or an SQL statement.
ConnectionString	Specifies the settings necessary to make a connection. For example, the connection string for Microsoft.Jet.OLEDB.3.51 provider is as follows: `"Provider=Microsoft.Jet.OLEDB.3.51; Data source=d:\\databases\\db1.mdb; Mode=Read|Write|ReadWrite"`

Notes

Events Associated with an ADO Data Control

The following table lists some ADO data control events that you can implement in your applications.

Event	Occurs
WillMove	When a recordset is opened or the current record of a recordset is changed.
MoveComplete	After the **WillMove** event.
WillChangeField	Before the **Value** property changes.
InfoMessage	When the data provider returns a result.

For a complete list of ADO data control properties and methods, search for "ADO Reference" in MSDN Library Visual Studio 6.0.

Notes

253

Data-bound Controls

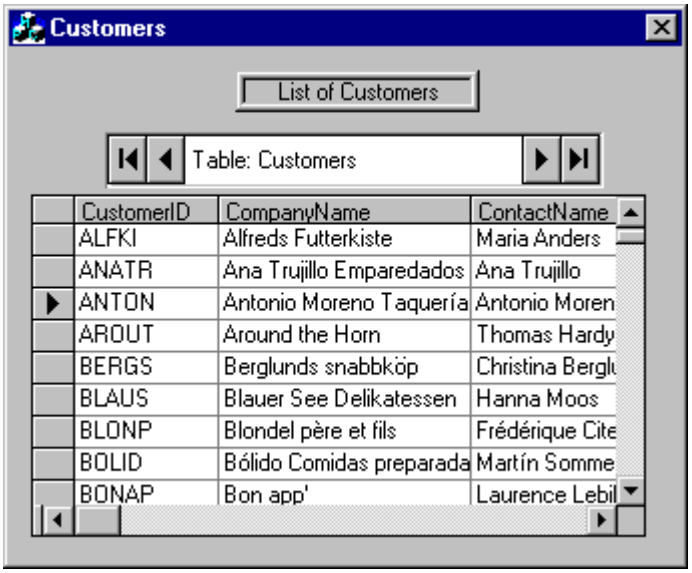
Data-bound controls are ActiveX controls that have two important characteristics:

- They have a **DataSource** property.
  You can set this property to the ID of an ADO data control.
- They can display data retrieved by the ADO data control to which they are bound.

ActiveX data controls include:

- **DataList**, **DataCombo**, and **DataGrid**.
- Lightweight controls, such as lightweight **CheckBox** and lightweight **ComboBox**.
- Microsoft FlexGrid and Hierarchical FlexGrid.

The following illustration shows a **DataGrid** control bound to an ADO data control.



For information about using a data-bound control, see Implementing ADO Using Data Controls in this chapter.

Notes

254 Implementing ADO Using Data Controls

To implement an ADO application that uses data controls, complete the following steps.

1. Create a dialog-based application.
2. Insert the ADO data control and set its properties.
3. Insert a data-bound control and set its properties.
4. Compile and run the program.

Each step is explained in more detail below.

To see a demonstration of how to create an ADO application that uses data controls, click this icon.
 (CD-ROM plays the demonstration, "Using Data Controls.")

▶ To create a dialog-based application

1. Invoke MFC AppWizard.
2. In Step 1 of MFC AppWizard, select **Dialog based**, and then click **Finish**.
3. Click **OK** to confirm the selection.

▶ To insert the ADO data control and set its properties

1. Invoke the Components and Controls Gallery and select **Registered ActiveX Controls**.

   A list of ActiveX controls appears.
2. Select **Microsoft ADO Data Control Version 6.0 (OLEDB)** and click **Insert**. Click **OK** to confirm the insertion.

   The **Confirm Classes** dialog box appears.
3. Click **OK**, and then **Close** to close the Gallery.
4. Invoke Resource Editor.

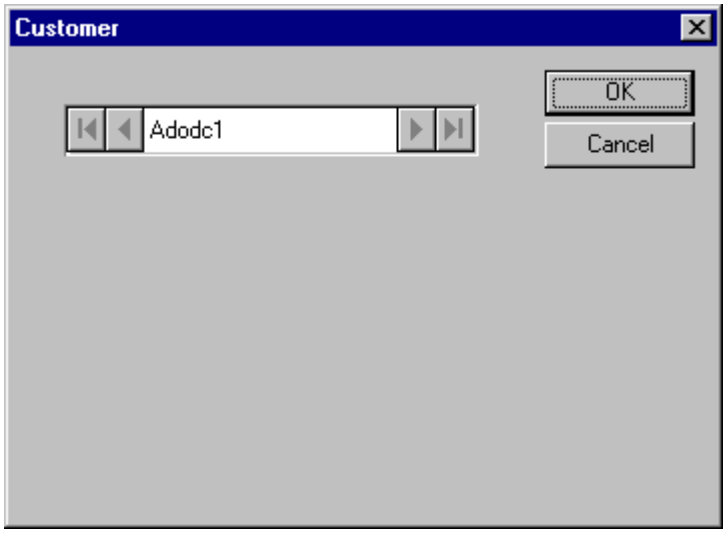
   Note the Microsoft ADO Data Control button on the **Controls** toolbar.

Notes

5. Delete the static control from the dialog box. Add an ADO data control to the dialog box.

   To see an illustration of the dialog box, click this icon.

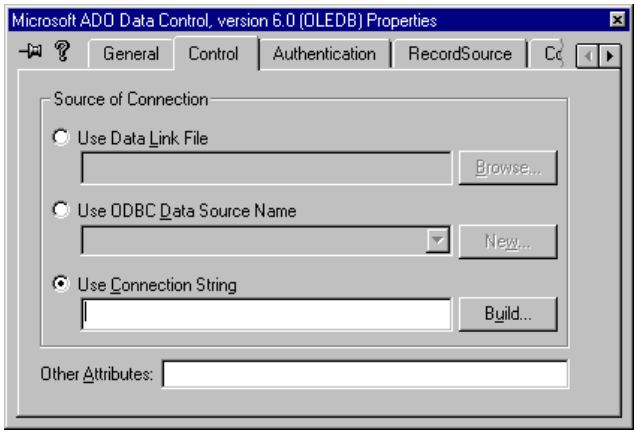
   (CD-ROM displays the following illustration.)



6. Right-click the data control and click **Properties** on the shortcut menu.

   The **Properties** dialog box appears. To see an illustration of the **Properties** dialog box, click this icon.

   (CD-ROM displays the following illustration.)

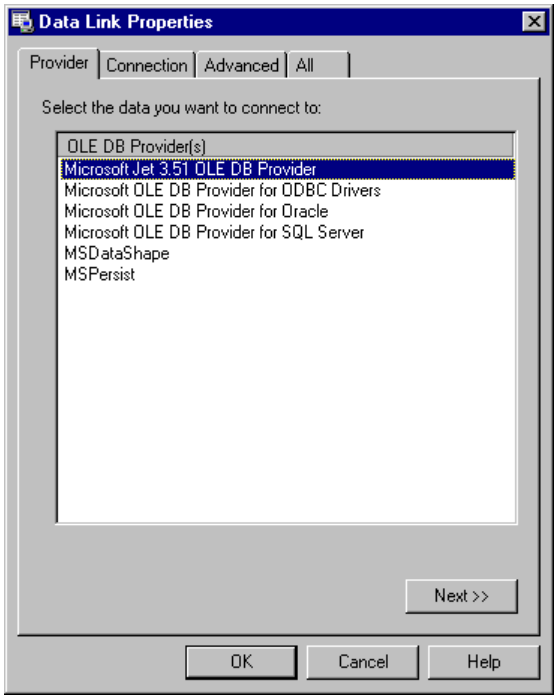


7. On the **General** tab, set a caption for the control; for example, **Table: Customer**.
8. On the **Control** tab, set the source of connection by selecting **Use Connection String**.

Notes

9. Click **Build** to construct the connection string.

   The **Data Link Properties** dialog box appears. To see an illustration of the **Data Link Properties** dialog box with the **Provider** tab selected, click this icon.
    (CD-ROM displays the following illustration.)

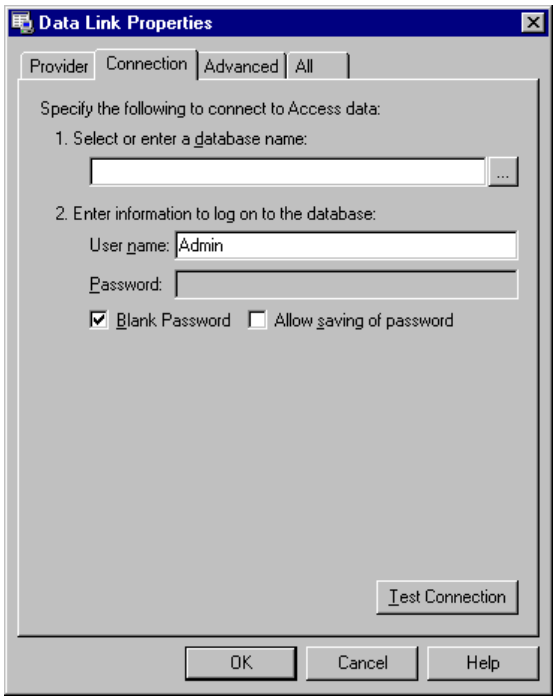


Notes

10. On the **Provider** tab, select a provider from the list (for example, **Microsoft Jet 3.51 OLE DB Provider**), and then click the **Connection** tab.

    To see an illustration of the **Data Link Properties** dialog box with the **Connection** tab selected, click this icon.

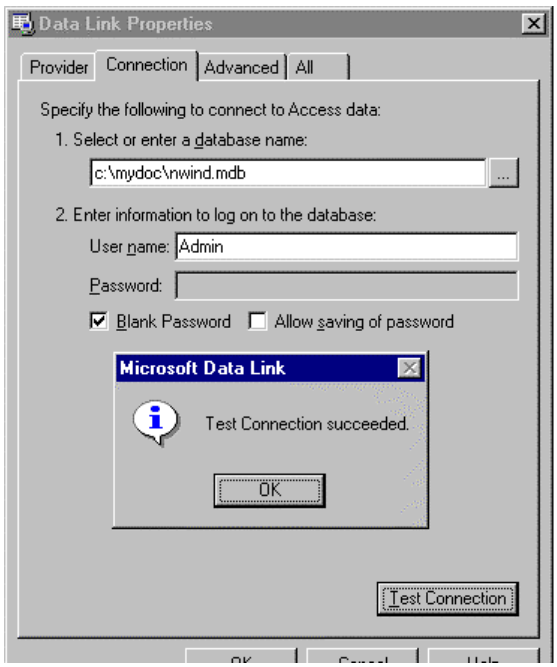
    (CD-ROM displays the following illustration.)



11. Specify the full path of the data source file name.
12. If the data source is password-protected, specify a user name and password to log on to the data source.

Notes

13. Click **Test Connection** to test the connection.
    To see an illustration of the results of testing a connection, click this icon.
    (CD-ROM displays the following illustration.)



14. Click **OK** to return to the Resource Editor.
    The complete connection string is displayed in the **Properties** dialog box. For example:
    `Provider=Microsoft.Jet.OLEDB.3.51;Data Source=C:\mydoc\nwind.mdb`
15. On the **RecordSource** tab, in the **Command Text** field, specify an SQL statement. For example:
    `SELECT * FROM CUSTOMERS`

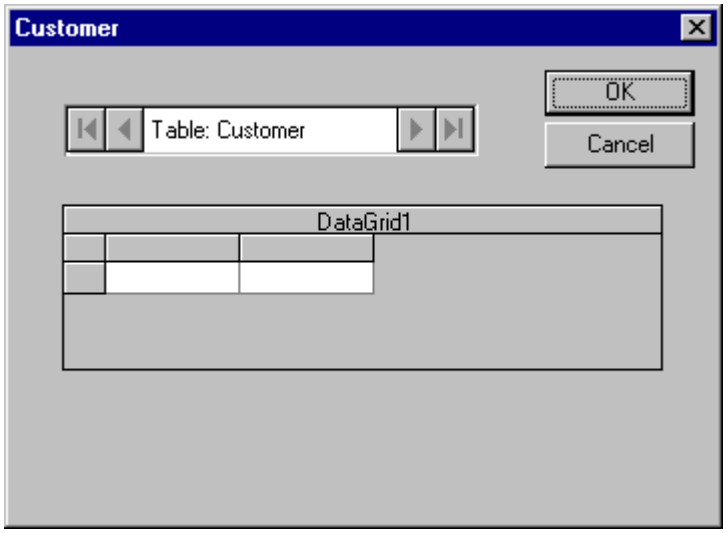
▶ **To insert a data-bound ActiveX control and set its properties**

1. Invoke the Components and Controls Gallery and select **Registered ActiveX Controls**.
   A list of ActiveX controls appears.
2. Select **Microsoft Data Grid Control, version 6.0** and click **Insert**. Click **OK** to confirm the insertion.
   The **Confirm Classes** dialog box appears.
3. Click **OK** to accept the default.
4. Click **Close** to close the Gallery.
5. Invoke Resource Editor.
   Note the **Microsoft Data Grid Control** button on the **Controls** toolbar.

Notes

6. Add the data grid control below the ADO data control.

   To see an illustration of the dialog box, click this icon.
   (CD-ROM displays the following illustration.)



7. Right-click the data-bound control and click **Properties** on the shortcut menu.
8. On the **All** tab, set the **DataSource** property to the ID of the ADO data control.

Compile and run the application. The recordset is displayed in the data grid control. Use the ADO data control to navigate through the recordset.

Notes

255 Using the Data-bound Dialog Box

In this section, you will learn how to implement ADO by creating a data-bound dialog box. The dialog box will contain intrinsic controls, such as edit boxes and check boxes, which are bound to the columns of a recordset.

You will learn to create resources, classes, and the Component Object Model (COM) initialization code necessary to implement the data-bound dialog box. The dialog box will allow record-level access to your data.

This section includes the following topics:

- Building an Application Using the Wizard
- Default Implementation of Data-bound Dialog Box
- Building an Application Without the Wizard
- Adding Records
- Editing Records
- Deleting Records

Notes

256 Building an Application Using the Wizard

The Enterprise Edition of Visual C++ 6.0 provides the ADO Data Bound Dialog Wizard to create skeletal code for a data-bound dialog box-based ADO application. However, if you are using the Professional Edition, this wizard is not available, and you will have to write the MFC AppWizard–generated code yourself. For information about how to do this, see Building an Application Without the Wizard in this chapter.

To implement an ADO application using the ADO Data Bound Dialog Wizard, complete the following steps.

1. Create a dialog-based application by using MFC AppWizard.
2. Invoke the ADO Data Bound Dialog Wizard to create the data-bound dialog box.
3. Add code to display the data-bound dialog box.

Each step is explained in more detail below.

To see a demonstration of how to create an ADO application by using the ADO Data Bound Dialog Wizard, click this icon.

 (CD-ROM plays the demonstration, "Using Data Bound Dialog Wizard.")

▶ **To create a dialog-based application by using MFC AppWizard**

1. Invoke MFC AppWizard. In Step 1 of MFC AppWizard, select **Dialog based**, and then click **Finish**.
2. Click **OK** to confirm the selection.

▶ **To invoke the Data Bound Dialog Wizard**

1. On the **Project** menu, click **Add To Project**, and then select **Components and Controls**.

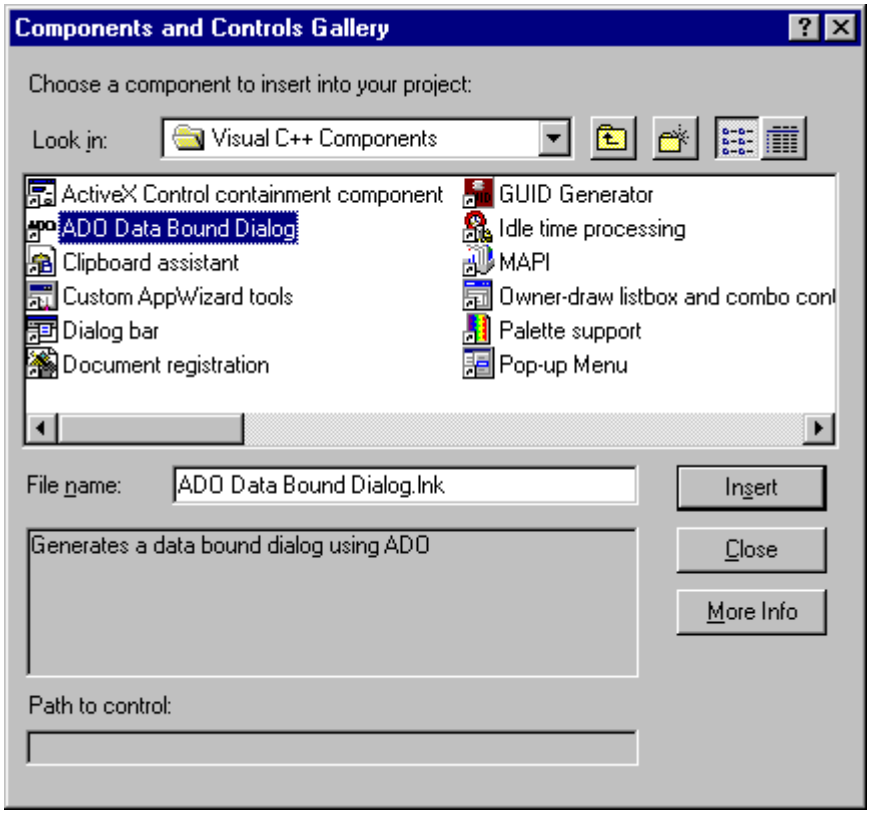
   The **Components and Controls Gallery** dialog box appears.
2. Double-click **Visual C++ Components**.

Notes

3. In the list of Visual C++ components, select **ADO Data Bound Dialog**.

   To see an illustration of the **Components and Controls Gallery** dialog box with the wizard selected, click this icon.

   (CD-ROM displays the following illustration.)

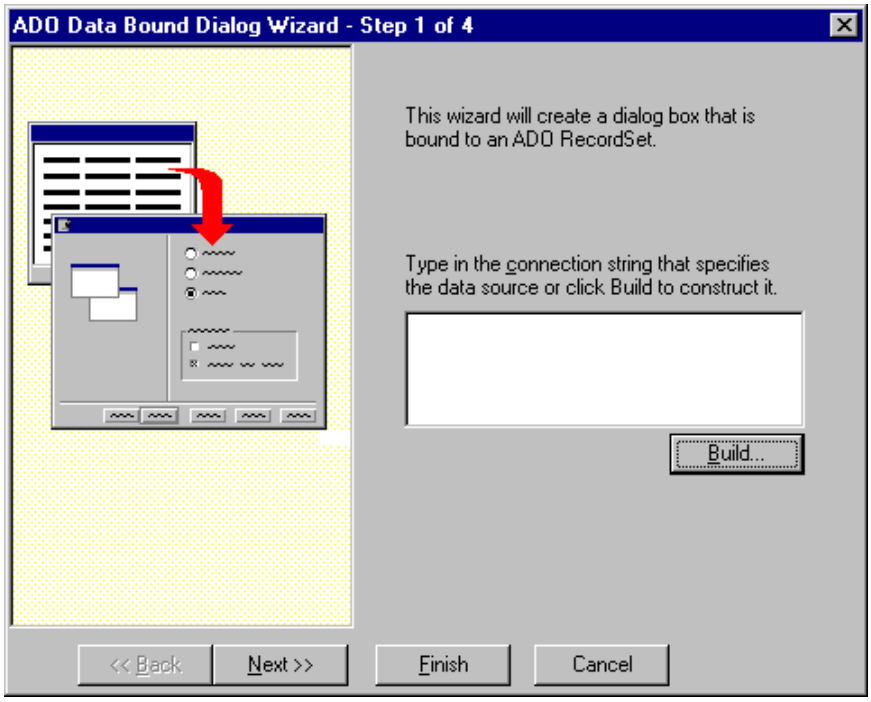


Notes

4. Click **Insert**, and then click **OK** to confirm the insertion.

   The **ADO Data Bound Dialog Wizard - Step 1 of 4** dialog box appears. To see an illustration of the **Step 1 of 4** dialog box, click this icon.

   (CD-ROM displays the following illustration.)

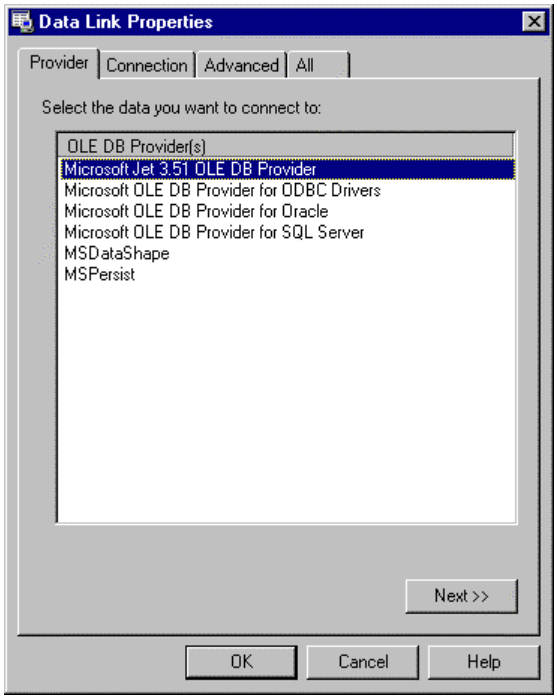


   You can type the connection string to specify the data source or click **Build** to build the string.

Notes

5. Click **Build** to build the connection string.

   The **Data Link Properties** dialog box appears. To see an illustration of the dialog box, click this icon. (CD-ROM displays the following illustration.)



6. On the **Provider** tab, choose a provider; for example, **Microsoft Jet 3.51 OLE DB Provider**.
7. On the **Connection** tab, specify the path to the data source and the user name and password to log on to the data source. Click **OK** to return to the wizard.

   The connection string will be similar to the following:

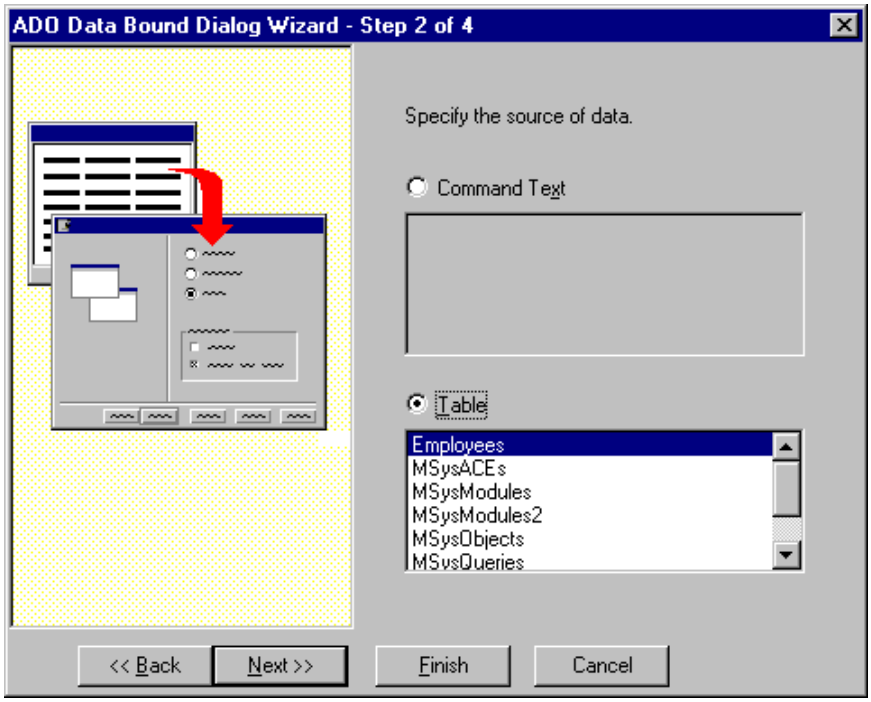
   ```
   Provider=Microsoft.Jet.OLEDB.3.51;Data Source=c:\databases\db1
   ```

Notes

8. Click **Next** to display the **ADO Data Bound Dialog Wizard - Step 2 of 4** dialog box.

   To see an illustration of the **Step 2 of 4** dialog box, click this icon.

   (CD-ROM displays the following illustration.)

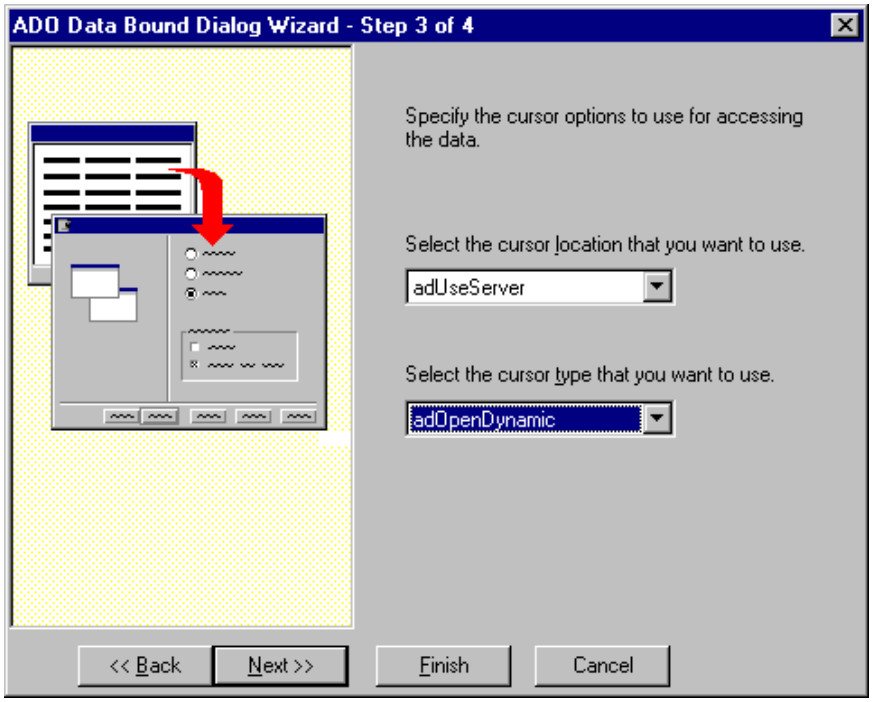


9. You can either select **Command Text** and type the SQL statement to retrieve the data or select **Table** and select the table name from the list that is displayed. Click **Table** and select a table from the list.

Notes

10. Click on **Next** to display the **ADO Data Bound Dialog Wizard - Step 3 of 4** dialog box.

    To see an illustration of the **Step 3 of 4** dialog box, click this icon.
    (CD-ROM displays the following illustration.)



11. Select a cursor location. The following options are available:
    - **adUseClient**

      Select **adUseClient** to use the MS Client Cursor Engine, which provides static, batch-updateable results.
    - **adUseServer**

      Select **adUseServer** to use cursors provided by the OLE DB provider.
12. Select a cursor type. The following options are available:
    - **adOpenStatic**

      Creates a static cursor. This cursor type provides you with a static copy of data that you can use to search or generate reports. Additions, changes, or deletions by other users are not visible.
    - **adOpenForwardOnly**

      Creates a forward-only cursor. This cursor type is similar to a static cursor except that you can only scroll forward through records. Improves performance in situations when you only need to make a single pass through a recordset.

Notes

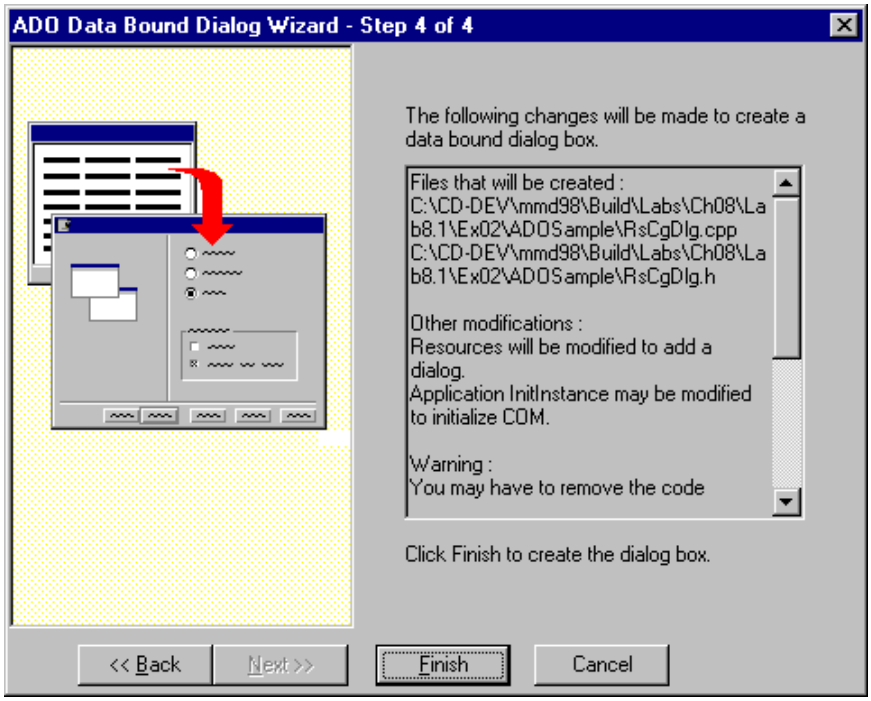
- **adOpenDynamic**

  Creates a dynamic cursor. Additions, changes, and deletions by other users are visible, and all types of movement through the recordset are allowed, except bookmarks if the provider doesn't support them.

- **adOpenKeyset**

  Creates a keyset cursor. This cursor type is similar to a dynamic cursor, except that you cannot see records that other users add, although records that other users delete are inaccessible from your recordset. Data changes by other users are made visible.

13. Click **Next** to display a list of changes that will be made to your project in **ADO Data Bound Dialog Wizard - Step 4 of 4**, or click **Finish** to return to the **Components and Controls Gallery** dialog box. To see an illustration of the **Step 4 of 4** dialog box, click this icon.
 (CD-ROM displays the following illustration.)



14. Click **Close** to close the Component and Controls Gallery.

▶ To add code to display the data-bound dialog box

You need to integrate the dialog box, **CG_IDD_RECORDSET**, with the dialog box created by MFC AppWizard. For example, you may add a button to the default dialog box that, when clicked, displays the data-bound dialog box.

Notes

The ADO Data Bound Dialog Wizard makes the following changes to your project:

- Adds the following classes:
  - **CCustomRs**

    This class is derived from **CADORecordBinding**. It binds the member variables to columns in the database. To see the definition and members of this class, click this icon.

     (CD-ROM displays the following sample code.)

```
// Samples\CH08\ADOSample2
// RsCgDlg.h
class CCustomRs :
    public CADORecordBinding
{
BEGIN_ADO_BINDING(CCustomRs)
    ADO_VARIABLE_LENGTH_ENTRY2(1, adVarChar, m_szAddress,
sizeof(m_szAddress), lAddressStatus, FALSE)
    ADO_VARIABLE_LENGTH_ENTRY2(2, adVarChar, m_szCity, sizeof(m_szCity),
lCityStatus, FALSE)
    ADO_VARIABLE_LENGTH_ENTRY2(3, adVarChar, m_szCompanyName,
sizeof(m_szCompanyName), lCompanyNameStatus, FALSE)
    ADO_VARIABLE_LENGTH_ENTRY2(4, adVarChar, m_szContactName,
sizeof(m_szContactName), lContactNameStatus, FALSE)
    ADO_VARIABLE_LENGTH_ENTRY2(5, adVarChar, m_szContactTitle,
sizeof(m_szContactTitle), lContactTitleStatus, FALSE)
    ADO_VARIABLE_LENGTH_ENTRY2(6, adVarChar, m_szCountry,
sizeof(m_szCountry), lCountryStatus, FALSE)
    ADO_VARIABLE_LENGTH_ENTRY2(7, adVarChar, m_szCustomerID,
sizeof(m_szCustomerID), lCustomerIDStatus, FALSE)
    ADO_VARIABLE_LENGTH_ENTRY2(8, adVarChar, m_szFax, sizeof(m_szFax),
lFaxStatus, FALSE)
    ADO_VARIABLE_LENGTH_ENTRY2(9, adVarChar, m_szPhone, sizeof(m_szPhone),
lPhoneStatus, FALSE)
    ADO_VARIABLE_LENGTH_ENTRY2(10, adVarChar, m_szPostalCode,
sizeof(m_szPostalCode), lPostalCodeStatus, FALSE)
    ADO_VARIABLE_LENGTH_ENTRY2(11, adVarChar, m_szRegion,
sizeof(m_szRegion), lRegionStatus, FALSE)
    END_ADO_BINDING()
```

(Code continues on following page.)

Notes

```
    protected:
        CHAR m_szAddress[61];
        ULONG lAddressStatus;
        CHAR m_szCity[16];
        ULONG lCityStatus;
        CHAR m_szCompanyName[41];
        ULONG lCompanyNameStatus;
        CHAR m_szContactName[31];
        ULONG lContactNameStatus;
        CHAR m_szContactTitle[31];
        ULONG lContactTitleStatus;
        CHAR m_szCountry[16];
        ULONG lCountryStatus;
        CHAR m_szCustomerID[6];
        ULONG lCustomerIDStatus;
        CHAR m_szFax[25];
        ULONG lFaxStatus;
        CHAR m_szPhone[25];
        ULONG lPhoneStatus;
        CHAR m_szPostalCode[11];
        ULONG lPostalCodeStatus;
        CHAR m_szRegion[16];
        ULONG lRegionStatus;
    };
```

(Code continues on following page.)

Notes

```
//
// TODO : These definitions should be moved to the resource header.
//
#define ID_BTN_MOVEPREVIOUS         201
#define ID_BTN_MOVENEXT             202
#define ID_BTN_MOVELAST             203
#define ID_BTN_MOVEFIRST            204
#define IDC_STATIC_ADDRESS      1001
#define IDC_EDIT_ADDRESS    2001
#define IDC_STATIC_CITY     1002
#define IDC_EDIT_CITY       2002
#define IDC_STATIC_COMPANYNAME      1003
#define IDC_EDIT_COMPANYNAME    2003
#define IDC_STATIC_CONTACTNAME      1004
#define IDC_EDIT_CONTACTNAME    2004
#define IDC_STATIC_CONTACTTITLE     1005
#define IDC_EDIT_CONTACTTITLE       2005
#define IDC_STATIC_COUNTRY      1006
#define IDC_EDIT_COUNTRY    2006
#define IDC_STATIC_CUSTOMERID       1007
#define IDC_EDIT_CUSTOMERID     2007
#define IDC_STATIC_FAX      1008
#define IDC_EDIT_FAX    2008
#define IDC_STATIC_PHONE    1009
#define IDC_EDIT_PHONE      2009
#define IDC_STATIC_POSTALCODE       1010
#define IDC_EDIT_POSTALCODE     2010
#define IDC_STATIC_REGION       1011
#define IDC_EDIT_REGION     2011
```

Note the BEGIN_ADO_BINDING macro, which binds the columns in the database to variables declared in the class. The wizard declares variables using data types that provide the closest match to the data types of the columns in the database.

Notes

- **CRsCgDlg**

  This class is derived from **CCustomRs** and **CDialog**. To see the definition and members of this class, click this icon.

   (CD-ROM displays the following sample code.)

```
// Samples\CH08\ADOSample2
// RsCgDlg.h
class CRsCgDlg :
    public CDialog,
    public CCustomRs
{
// Construction
public:
    CRsCgDlg(CWnd* pParent = NULL);
    ~CRsCgDlg();

    BOOL OnInitDialog();

// Dialog Data
    //{{AFX_DATA(CRsCgDlg)
    enum { IDD = CG_IDD_RECORDSET };
    CString m_strDlgAddress;
    CString m_strDlgCity;
    CString m_strDlgCompanyName;
    CString m_strDlgContactName;
    CString m_strDlgContactTitle;
    CString m_strDlgCountry;
    CString m_strDlgCustomerID;
    CString m_strDlgFax;
    CString m_strDlgPhone;
    CString m_strDlgPostalCode;
    CString m_strDlgRegion;
    //}}AFX_DATA


// Overrides
    // ClassWizard generated virtual function overrides
    //{{AFX_VIRTUAL(CRsCgDlg)
    protected:
    virtual void DoDataExchange(CDataExchange* pDX);    // DDX/DDV support
    //}}AFX_VIRTUAL
```

(Code continues on following page.)

Notes

```
// Implementation
protected:

    void RefreshBoundData();
    void GenerateError(HRESULT hr, PWSTR pwszDescription);

    _RecordsetPtr m_pRs;

    CString m_strConnection;
    CString m_strCmdText;

    // Generated message map functions
    //{{AFX_MSG(CRsCgDlg)
    afx_msg void OnBtnMoveFirst();
    afx_msg void OnBtnMoveLast();
    afx_msg void OnBtnMoveNext();
    afx_msg void OnBtnMovePrevious();
    //}}AFX_MSG
    DECLARE_MESSAGE_MAP()
};
```

- Adds the following files:
  - RsCgDlg.h
  - RsCgDlg.cpp

  If your project contains multiple data-bound dialog boxes, these file names are appended with a number starting with 1. For example, the files corresponding to the second data-bound dialog box in your project would be named RsCgDlg1.cpp and RsCgDlg1.h.
- Adds a dialog box named **CG_IDD_RECORDSET** with controls to display the data values.
- Adds code to the project's application **InitInstance** to initialize COM. If you added COM initialization code to your application manually, then you may need to remove the code added by the ADO Data Bound Dialog Wizard.

Notes

257 Default Implementation of Data-bound Dialog Box

The RsCgDlg.cpp file implements the functionality of the data-bound dialog box.

The constructor of the dialog box initializes two strings—the connection string and the command text.

Connection String

The connection string stores information about the type of provider being used, the data source, and the mode in which the data source is to be opened. If the provider is Microsoft.Jet.OLEDB.3.51, then the data source is specified as the path to the database file. If the provider is ODBC, the data source is the name of the ODBC data source.

The following code shows the connection string for an ODBC provider:

```
m_strConnection = _T("Provider=MSDASQL.1;User ID=admin;Data source=ado3;
Mode=Read|Write|ReadWrite");
```

The following code shows the connection string for the Microsoft.Jet.OLEDB.3.51 provider:

```
m_strConnection = _T("Provider=Microsoft.Jet.OLEDB.3.51; Data
source=d:\\databases\\db1.mdb; Mode=Read|Write|ReadWrite");
```

Command Text

The command text can either be the name of a table or an SQL statement as shown in the following example:

```
m_strCmdText = _T("Employees");
m_strCmdText = _T("Select * from Employees");
```

In the **OnInitDialog** function, **CreateInstance** function is invoked to create a pointer of type **_RecordsetPtr**, as shown in the following example:

```
\\ m_pRs is declared as _RecordsetPtr m_pRs;
m_pRs.CreateInstance(_uuidof(Recordset));
```

The CursorLocation attribute is set to specify the location of the cursor. For example:

```
m_pRs->CursorLocation = adUseServer;
```

The Open Function

The **Open** function is called to open a cursor for the recordset, as shown in the following example:

```
m_pRs->Open((LPCTSTR)m_strCmdText, (LPCTSTR)m_strConnection, adOpenStatic,
adLockOptimistic, adCmdTableDirect);
```

The **Open** function takes the following arguments:

- Command text

  Specifies the command text.
- Connection string

  Specifies the connection string.

Notes

- Cursor type

  Defines the type of cursor to be created.
- Locking type to be used with the cursor

  You can choose one of the following values for this argument.

Lock type	Description
adLockReadOnly	Read-only—you cannot alter the data.
adLockPessimistic	Pessimistic locking (by record or by page depending on the provider)—the provider does what is necessary to ensure successful editing of the records, usually by locking records at the data source immediately upon editing.
adLockOptimistic	Optimistic locking (by record or by page depending on the provider)—the provider uses optimistic locking, locking records only when you call the **Update** method.
adLockBatchOptimistic	Optimistic batch updates—required for batch update mode as opposed to immediate update mode**.**

- Type of command object

  You can choose one of the following values for this argument.

Command type option	Description
adCmdText	Evaluates **CommandText** as a textual definition of a command.
adCmdTable	Evaluates **CommandText** as a table name in a generated SQL query returning all columns.
adCmdTableDirect	Evaluates **CommandText** as a table name whose columns are all returned.
adCmdStoredProc	Evaluates **CommandText** as a stored procedure.
adCmdUnknown	Default. The type of command in the **CommandText** property is not known.
adCommandFile	Evaluates **CommandText** as the file name of a persisted recordset.

The QueryInterface Function

The call to **Open** is followed by a call to the **QueryInterface** function. This function returns a pointer to the IADORecordBinding interface. It returns S_OK if the interface is supported, or S_FALSE if the interface is not supported.

The following example shows the **QueryInterface** function initializing the **piAdoRecordBinding** pointer:

```
m_pRs->QueryInterface(__uuidof(IADORecordBinding), (LPVOID
*)&piAdoRecordBinding);
```

Notes

The keyword __**uuidof** retrieves the GUID attached to the expression IADORecordBinding. The pointer piAdoRecordBinding is of type IADORecordBinding and receives the pointer to the interface, if it is supported.

The member function **BindToRecordset** of the IADORecordBinding interface is called to associate the fields in the rowset with variables as shown in the following sample code:

```
piAdoRecordBinding->BindToRecordset(this);
```

To see the complete code for the **InitDialog** function, click this icon.
 (CD-ROM displays the following sample code.)

```
//Samples\CH08\ADOSample2
BOOL CRsCgDlg::OnInitDialog()
{
    IADORecordBinding *piAdoRecordBinding = NULL;

    CDialog::OnInitDialog();

    try
    {
        m_pRs.CreateInstance(__uuidof(Recordset));

        m_pRs->CursorLocation = adUseServer;
        m_pRs->Open((LPCTSTR)m_strCmdText, (LPCTSTR)m_strConnection,
adOpenStatic,
            adLockOptimistic, adCmdTableDirect);

        if (FAILED(m_pRs->QueryInterface(__uuidof(IADORecordBinding), (LPVOID
*)&piAdoRecordBinding)))
            _com_issue_error(E_NOINTERFACE);
        piAdoRecordBinding->BindToRecordset(this);
        RefreshBoundData();
    }
    catch (_com_error &e)
    {
        GenerateError(e.Error(), e.Description());
    }

    if (piAdoRecordBinding)
        piAdoRecordBinding->Release();

    return TRUE;
}
```

Notes

Message Handlers

The ADO Data Bound Dialog Wizard defines message handlers to navigate through the records. The following table lists these message handlers and describes the functions they invoke.

Message handler	Function invoked	Recordset pointer moves to
OnBtnMoveFirst	**MoveFirst**	First record
OnBtnMovePrevious	**MovePrevious**	Previous record
OnBtnMoveNext	**MoveNext**	Next record
OnBtnMoveLast	**MoveLast**	Last record

These handlers also invoke a function named **RefreshBoundData**, which transfers the data from the recordset to the dialog box controls. The following code shows an example of a **RefreshBoundData** function:

```
void CRsCgDlg::RefreshBoundData()
{
    if (adFldOK == lEmployeeIDStatus)
        m_lDlgEmployeeID = m_lEmployeeID;
    else
        m_lDlgEmployeeID = _T("");
    if (adFldOK == lFirstNameStatus)
        m_strDlgFirstName = m_szFirstName;
    else
        m_strDlgFirstName = _T("");
    if (adFldOK == lLastNameStatus)
        m_strDlgLastName = m_szLastName;
    else
        m_strDlgLastName = _T("");

    UpdateData(FALSE);
}
```

If the data source returns no records, the provider sets both the **BOF** and **EndOfFile** properties to **TRUE**, and the current record position is undefined. If you selected an updateable cursor type, you can still add new data to this empty recordset object.

Notes

258 Building an Application Without the Wizard

With the Enterprise Edition of Visual C++ 6.0, you can use the Data Bound Dialog Wizard to create skeletal code that displays the records in a database table. However, if you are using the Professional edition of Visual C++ 6.0, you have to write code to implement the same functionality. To create a data-bound dialog box without using the wizard, you follow these steps:

1. Create a dialog-based application using MFC AppWizard.
2. Add controls to the default dialog box.
3. Modify your application to initialize OLE DLLs.
4. Modify the dialog header file to ensure that the application works with newer versions of ADO.
5. Create a **CADORecordBinding**-derived class to store the records that are accessed from the database table.
6. Modify the default dialog class, which is created by MFC AppWizard, to implement access to the database.
7. Add handlers to implement scrolling through the records in the recordset.

These steps are described in more detail below.

▶ **To create a dialog-based application using MFC AppWizard**

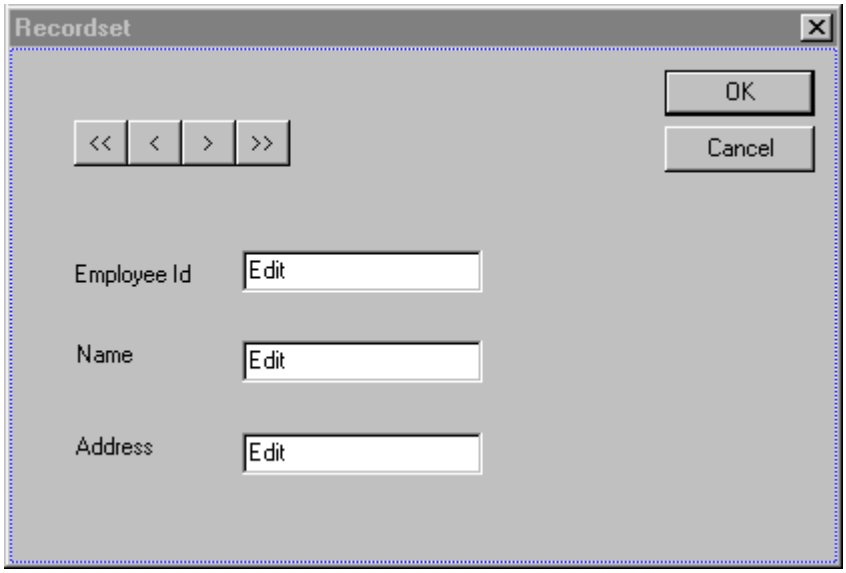
- Create a new project using MFC AppWizard and in the **Step 2 of 6** dialog box, click **Dialog** and click **Finish**.

▶ **To add controls to the default dialog box**

1. From the **Controls** toolbar, drag suitable controls onto the dialog box for each field in the database table being accessed.
2. Add suitable prompts for each field.
3. Add controls to the dialog box to implement scrolling through the database table.

Notes

To see an illustration of a sample dialog box, click this icon.
(CD-ROM displays the following illustration.)



▶ **To modify your application to initialize OLE DLLs**

1. Open the **CWinApp** implementation file.
2. In the **InitInstance** function, call the **AfxOleInit** function to initialize OLE libraries, as shown in the following example:

```
if (!AfxOleInit())
{
    AfxMessageBox(_T("OLE initialization failed."));
    return FALSE;
}
```

▶ **To modify the dialog header file to ensure that the application works with newer versions of ADO**

- Import the ADO library file MSADO15.DLL and include the header file icrsint.h, as shown in the following example:

```
#import "C:\PROGRAM FILES\COMMON FILES\SYSTEM\ADO\MSADO15.DLL"
rename_namespace("ADOCG") rename("EOF", "EndOfFile")
using namespace ADOCG;
#include "icrsint.h"
```

  This ensures that the code can be used with different versions of ADO.

Notes

▶ **To create a CADORecordBinding-derived class to store the records that are accessed from the database table**

1. In the header file of the dialog class, create a **CADORecordBinding**-derived class.
2. Add protected member variables to the class for each field in the database table, as shown in the following example:

```
protected:
    LONG m_lID;
    CHAR m_szName[51];
    CHAR m_szAddress[51];
```

   The data types of the protected member variables should match the data types of the fields in the database table.

3. Bind the member variables defined in the previous step to the fields in the database table by using the preprocessor macro BEGIN_ADO_BINDING. To see a sample **CADORecordBinding**-derived class, click this icon.


(CD-ROM displays the following sample code.)

```
//Samples\CH08\ADOSample3
//Stores the records that are retrieved from a table
class CCustomRs :
    public CADORecordBinding
{
BEGIN_ADO_BINDING(CCustomRs)
    ADO_FIXED_LENGTH_ENTRY(1, adInteger, m_lID, lIDStatus, FALSE)
    ADO_VARIABLE_LENGTH_ENTRY2(2, adVarChar, m_szName, sizeof(m_szName),
lNameStatus, FALSE)
    ADO_VARIABLE_LENGTH_ENTRY2(3, adVarChar, m_szAddress,
sizeof(m_szAddress), lAddressStatus, FALSE)
END_ADO_BINDING()

protected:
    LONG m_lID;
    ULONG lIDStatus;
    CHAR m_szName[51];
    ULONG lNameStatus;
    CHAR m_szAddress[51];
    ULONG lAddressStatus;
};
```

In the previous sample code, the macro invokes the ADO_FIXED_LENGTH_ENTRY and ADO_VARIABLE_LENGTH_ENTRY2 macro functions.

The ADO_FIXED_LENGTH_ENTRY macro function sets up the binding between the fixed-length entry columns, such as integers and the member variables.

The ADO_VARIABLE_LENGTH_ENTRY2 macro function is invoked to set up the binding between the variable-length entry columns, such as character strings and the member variables.

Notes

Both ADO_FIXED_LENGTH_ENTRY and ADO_VARIABLE_LENGTH_ENTRY2 take the following arguments:

- Column to bind to
- Data type of the variable where the converted field will be stored
- Buffer used to convert the field to a variable

  With ADO_VARIABLE_LENGTH_ENTRY2, you also have to specify the size of the buffer as the fourth argument.
- Return value that indicates success of the field conversion

  The value returned is stored in a variable of type **ULONG** that must be declared within the class for each field. With ADO_VARIABLE_LENGTH_ENTRY2, this is the fifth argument.
- Boolean flag

  A value of **TRUE** indicates that ADO can update the associated field. With ADO_VARIABLE_LENGTH_ENTRY2, this is the sixth argument.

▶ **To implement the default dialog class generated by MFC AppWizard**

1. Modify the dialog class declaration to derive from the **CDialog** and the **CCustomRs** classes, as shown in the following example:

   ```
   class CADOSample1Dlg : public CDialog, public CCustomRs
   {
       .....
       .....
   }
   ```
2. Add two protected member variables of type **CString** to the dialog class, as shown in the following example:

   ```
   CString m_strConnection;
   CString m_strCmdText;
   ```

   The variable m_strConnection is for storing the connection string, and the variable strCmdText is for storing the name of the table to be opened or the SQL statement to be used to access the database table.
3. Using ClassWizard, declare member variables for each control. The controls will be used to display data from the database table.
4. Modify the constructor of the dialog class to set the connection string and command text variables, as shown in the following example:

   ```
   CADOSample1Dlg::CADOSample1Dlg(CWnd* pParent /*=NULL*/)
       : CDialog(CADOSample1Dlg::IDD, pParent)
   {
       m_strConnection = _T("Provider=MSDASQL.1;Data Source=OLEDBSample");
       m_strCmdText = _T("Employees");
       .......
   }
   ```

   The connection string is a semicolondelimited (;) string that specifies the provider and the data source to be used.

Notes

5. Add a protected _RecordsetPtr type variable to the dialog class, as shown in the following example:

```
class CADOSample1Dlg :
    public CDialog,
    public CCustomRs
{
.....
protected:

    _RecordsetPtr m_pRs;
.....
}
```

6. Set the _RecordsetPtr type object to **NULL** in the dialog constructor and the destructor.
7. Create a member function named **RefreshBoundData** to transfer the contents of the bound member variables of the **CCustomRs** class to the member variables associated with the controls on the dialog box. To see sample code for the **RefreshBoundData** function, click this icon.

 (CD-ROM displays the following sample code.)

```
//Samples\CH08\ADOSample3
//Transfers data from the recordset to the dialog member variables
void CADOSample1Dlg::RefreshBoundData()
{
    if (adFldOK == lIDStatus)
        m_lDlgID = m_lID;
    else
        m_lDlgID = 0;
    if (adFldOK == lNameStatus)
        m_strDlgName = m_szName;
    else
        m_strDlgName = _T("");
    if (adFldOK == lAddressStatus)
        m_strDlgAddress = m_szAddress;
    else
        m_strDlgAddress = _T("");

    UpdateData(FALSE);
}
```

8. In the **OnInitDialog**-derived function, declare a pointer of type IADORecordBinding and set it to **NULL**, as shown in the following example:

```
BOOL CADOSample1Dlg::OnInitDialog()
{
    IADORecordBinding *piAdoRecordBinding = NULL;
    .....
    .....
}
```

9. Create an instance of the **_RecordsetPtr** object using the **CreateInstance** member function, as shown in the following example:

```
m_pRs.CreateInstance(__uuidof(Recordset));
```

Notes

10. Set the cursor location using the _RecordsetPtr member variable CursorLocation, as shown in the following example:

```
m_pRs->CursorLocation = adUseServer;
```

11. Open _RecordsetPtr using the **Open** member function, as shown in the following example:

```
m_pRs->Open((LPCTSTR)m_strCmdText, (LPCTSTR)m_strConnection, adOpenDynamic,
adLockOptimistic, adCmdTableDirect);
```

    The **Open** function takes the following arguments:

    - Command text

      This could either be an SQL statement or the name of the table to be opened.

    - Connection string

      This specifies the provider and the data source to be used.

    - Type of cursor to be used
    - Type of locking
    - Type of command object

12. Bind the recordset to the dialog box by completing the following steps:

    - Invoke the **QueryInterface** function to get a pointer to the IADORecordBinding interface, as shown in the following example:

```
if (FAILED(m_pRs->QueryInterface(__uuidof(IADORecordBinding), (LPVOID
*)&piAdoRecordBinding)))
    _com_issue_error(E_NOINTERFACE);
```

    - Invoke the **BindToRecordset** function to bind the recordset to the dialog box as follows:

```
piAdoRecordBinding->BindToRecordset(this);
```

13. Invoke the **RefreshBoundData** function to transfer the data from the recordset to the controls on the dialog box.

The functions that are used to create an instance of the _RecordsetPtr object, open the _RecordsetPtr object, and bind the recordset to the dialog box can be placed within a **try** block. A suitable error handler can be invoked in the **catch** block to handle any error that may occur.

Notes

To see the complete code for the **OnInitDialog** function, click this icon.
 (CD-ROM displays the following sample code.)

```
//Samples\CH08\ADOSample3
//Opens the recordset and binds the recordset to the columns of the
//data source
BOOL CADOSample1Dlg::OnInitDialog()
{
    IADORecordBinding *piAdoRecordBinding = NULL;

    CDialog::OnInitDialog();

    try
    {
        m_pRs.CreateInstance(__uuidof(Recordset));

        m_pRs->CursorLocation = adUseClient;
        m_pRs->Open((LPCTSTR)m_strCmdText, (LPCTSTR)m_strConnection,
adOpenStatic,
            adLockReadOnly, adCmdTableDirect);

        if (FAILED(m_pRs->QueryInterface(__uuidof(IADORecordBinding), (LPVOID
*)&piAdoRecordBinding)))
            _com_issue_error(E_NOINTERFACE);
        piAdoRecordBinding->BindToRecordset(this);

        RefreshBoundData();
    }
    catch (_com_error &e)
    {
        GenerateError(e.Error(), e.Description());
    }

    if (piAdoRecordBinding)
        piAdoRecordBinding->Release();

    return TRUE;
}
```

Notes

▶ **To implement scrolling through the records in the recordset**

- Create handlers to implement move-first, move-next, move-previous, and move-last functionality. To see sample code for the **OnMoveNextFunction**, click this icon.

   (CD-ROM displays the following sample code.)

```
//Samples\CH08\ADOSample3
//Displays the next record in the recordset.
void CADOSample1Dlg::OnBtnMoveFirst()
{
    try
    {
        m_pRs->MoveFirst();
        RefreshBoundData();
    }
    catch (_com_error &e)
    {
        GenerateError(e.Error(), e.Description());
    }
}
```

  In the previous sample code, the _RecordsetPtr member function **MoveFirst** was invoked to scroll to the first record. You can also invoke **MoveNext**, **MovePrevious**, and **MoveLast** to reposition the current record pointer. After scrolling to a new record, you invoke the **RefreshBoundData** function to refresh the controls of the dialog box. The statements are placed within a **try** block to avoid scrolling beyond end of file or beginning of file.

Notes

259 Adding Records

You can add records to an updateable recordset by calling the **AddNew** function of the **_Recordset** class. The **AddNew** function creates and initializes a new record. The function takes two arguments, both of which are safearrays of variants. The first argument contains the names of the columns in the database. The second argument contains the data values for each column.

To see sample code that invokes the **AddNew** function to add a new record to the database, click this icon.

 (CD-ROM displays the following sample code.)

```
// Adds a new record to a recordset
// Samples\CH08\ADOSample3
void CADOSample3Dlg::OnAddRecord()
{
    long rgIndices[1];
    VARIANT rgf; //Stores field names
    VARIANT rgv; //Stores field values
    COleVariant fld[3], val[3];
    SAFEARRAYBOUND bound;
    try
    {
        UpdateData(TRUE);
        fld[0]="EmployeeID";
        val[0]=m_lDlgID;
        fld[1]="Name";
        val[1]=m_strDlgName;
        fld[2]="Address";
        val[2]=m_strDlgAddress;

        rgf.vt=VT_ARRAY|VT_VARIANT;
        rgv.vt=VT_ARRAY|VT_VARIANT;

        bound.cElements=3;
        bound.lLbound=0;
```

(Code continues on following page.)

Notes

```
        //Create a safe array of VARIANTs to store the field values
        rgv.parray=::SafeArrayCreate(VT_VARIANT,1,&bound);
        if(rgv.parray==NULL)
            AfxThrowMemoryException();
        //Create a safe array of VARIANTs to store the field names
        rgf.parray=::SafeArrayCreate(VT_VARIANT,1,&bound);
        if(rgf.parray==NULL)
            AfxThrowMemoryException();
        //Store the field names and values in the safearray of VARIANTs
        for ( int i = 0; i < 3; i++)
        {
            rgIndices[0]=i;
            ::SafeArrayPutElement(rgf.parray, rgIndices, &fld[i]);
            ::SafeArrayPutElement(rgv.parray, rgIndices, &val[i]);
        }

        m_pRs->AddNew(&rgf,&rgv);
    }
    catch (_com_error &e)
    {
        GenerateError(e.Error(), e.Description());
    }
}
```

Notes

260 Editing Records

You use the **Update** function to edit the current record of an updateable recordset. You pass the names of the fields to be updated and the new values to the **Update** function.

To see sample code that invokes the **Update** function to update the current record of the recordset, click this icon.

 (CD-ROM displays the following sample code.)

```
// Updating the current record of a recordset
// Samples\CH08\ADOSample3
void CADOSample3Dlg::OnEditRecord()
{
    long rgIndices[1];
    VARIANT rgf; //Stores field names
    VARIANT rgv; //Stores field values
    COleVariant fld[3], val[3];
    SAFEARRAYBOUND bound;

    try
    {
        UpdateData(TRUE);
        fld[0]="EmployeeID";
        val[0]=m_lDlgID;
        fld[1]="Name";
        val[1]=m_strDlgName;
        fld[2]="Address";
        val[2]=m_strDlgAddress;

        rgf.vt=VT_ARRAY|VT_VARIANT;
        rgv.vt=VT_ARRAY|VT_VARIANT;

        bound.cElements=3;
        bound.lLbound=0;
```

(Code continues on following page.)

Notes

```
        //Create a safe array of VARIANTs to store the field values
        rgv.parray=::SafeArrayCreate(VT_VARIANT,1,&bound);
        if(rgv.parray==NULL)
            AfxThrowMemoryException();
        //Create a safe array of VARIANTs to store the field names
        rgf.parray=::SafeArrayCreate(VT_VARIANT,1,&bound);
        if(rgf.parray==NULL)
            AfxThrowMemoryException();
        //Store the field names and values in the safearray of VARIANTs
        for ( int i = 0; i < 3; i++)
        {
            rgIndices[0]=i;
            ::SafeArrayPutElement(rgf.parray, rgIndices, &fld[i]);
            ::SafeArrayPutElement(rgv.parray, rgIndices, &val[i]);
        }
        m_pRs->Update(&rgf,&rgv);
    }
    catch (_com_error &e)
    {
        GenerateError(e.Error(), e.Description());
    }
}
```

Notes

261 Deleting Records

Use the **Delete** function to delete the current record of a recordset. You can also delete a group of records using this function.

The **Delete** function takes a single argument. The value of this argument can either be **adAffectCurrent** or **adAffectGroup**. The default, **adAffectCurrent**, deletes only the current record, while **adAffectGroup** deletes the records that satisfy the current filter property setting. To use **adAffectGroup**, you must set the filter property to one of the valid predefined constants.

To see sample code that deletes the current record of the recordset, click this icon.
 (CD-ROM displays the following sample code.)

```
//Deleting the current record of a recordset
// Samples\CH08\ADOSample3
void CADOSample3Dlg::OnDelRecord()
{
        try
        {
            m_pRs->Delete(adAffectCurrent);
            m_pRs->MoveNext();
            if(m_pRs->EndOfFile)
                m_pRs->MoveLast();
            RefreshBoundData();

        }
        catch (_com_error &e)
        {
            if(m_pRs->EndOfFile && m_pRs->BOF)
            {
                BlankForm();
                UpdateData(FALSE);
            }
            else
                GenerateError(e.Error(), e.Description());
        }
}
```

Notes

262 Performing Queries

In this section, you will learn how to perform performing queries on data sources and rowsets. You will also learn how to search for specific records in a rowset.

This section includes the following topics:

- Performing a Simple Query
- Using Filters
- Searching for Records

Notes

263 Performing a Simple Query

You perform queries on a database by altering the WHERE clause associated with the recordset. To specify or change the selection criteria, complete the following steps:

1. Close the recordset object by invoking the **Close** member function. Closing the recordset object does not remove it from memory.
2. Change the property settings of the recordset.
3. Reopen the recordset.

The following example shows how to perform a simple query to retrieve those records whose Employee ID is greater than 3:

```
m_pRs->Close();
m_strCmdText = _T("Select * from employees where EmployeeID < 3");
m_pRs->Open((LPCTSTR)m_strCmdText, (LPCTSTR)m_strConnection, adOpenStatic,
adLockReadOnly, adCmdUnknown);
if (FAILED(m_pRs->QueryInterface(__uuidof(IADORecordBinding),    (LPVOID
*)&piAdoRecordBinding)))
    _com_issue_error(E_NOINTERFACE);
piAdoRecordBinding->BindToRecordset(this);
```

Notes

264 Using Filters

You set the **Filter** property of the recordset to filter records in the recordset that satisfy a search criterion. When constructing the search criteria, you can use comparison operators such as "**>**" (greater than), "**<**" (less than), "**=**" (equal), or "**like**" (pattern matching). The comparison operator **like** can be used in conjunction with the asterisk (*****) wildcard, which represents zero or more characters.

The following example selects all records whose **LastName** starts with "M."

```
m_pRs->Filter = "LastName like 'M*'";
```

You can also use logical operators such as **AND** and **OR** in the filter criteria, as shown in the following example:

```
m_pRs->Filter = "LastName like 'M*' OR LastName like 'S*'";
```

There is no precedence between **AND** and **OR**. Clauses can be grouped within parentheses. However, you cannot group clauses joined by **OR** with another clause by using **AND**. For example, you cannot group clauses as follows:

```
m_pRs->Filter = "(LastName like 'S*' OR LastName like 'M*') AND FirstName like
'D*'";
```

Instead, you would construct this filter as:

```
m_pRs->Filter = "(LastName Like 'S*' AND FirstName like 'D*') OR (LastName
like 'M*' AND FirstName like 'D*')";
```

Notes

265 Searching for Records

You use the **Find** member function to search a recordset for records that satisfy the specified criteria. When a record that satisfies the search criteria is found, it becomes the current record. If none of the records in the recordset satisfy the search criteria, the current position is set to end of the recordset.

The following example shows how you can search for records within a recordset:

```
m_pRs->Find("LastName like 'M*'",3,adSearchForward);
```

The **Find** function takes the following arguments:

- A string containing the search criteria
- An optional long value that defines the starting point of the search operation

  You specify the starting location relative to the current record. For example, if you specify 3, the search skips three records from the current record.
- The search direction, which can be either **adSearchForward** or **adSearchBackward**
- An optional bookmark

  If you specify a bookmark, it will be used as the starting position for the search.

Notes

267 Sample Applications

The following table describes the sample applications related to this chapter. These sample applications are located in the folder \Samples\Ch08.

Sample application subfolder	Description of application
\ADOSample1	An application that uses the ADO data control with a data-bound control to access data in a database table. Ensure that the database file emp.mdb exists in the directory from which the application is executed, and has read-write permissions.
\ADOSample2	An application that uses the ADO data-bound dialog box to access data in a database table. The application has been created using the Data Bound Dialog Wizard available with the Enterprise edition of Visual C++ 6.0. Ensure that the database file emp.mdb exists in the directory from which the application is executed, and has read-write permissions.
\ADOSample3	An application that uses the ADO data-bound dialog box to access data in a database table. The application has been created without using the Data Bound Dialog Wizard. Ensure that the database file emp.mdb exists in the directory from which the application is executed, and has read-write permissions.

Notes

269

Self-Check Questions

1. Which object contains information on the data provider and the underlying schema?

- A. **Connection**
- B. **Recordset**
- C. **Command**
- D. **Field**

2. Which object contains an Errors collection created in response to a failure involving the provider?

- A. **Connection**
- B. **Command**
- C. **Recordset**
- D. **Parameters**

3. Which edition of Visual C++ provides the ADO Data Bound Dialog Wizard?

- A. **Professional only**
- B. **Enterprise only**
- C. **Both Professional and Enterprise**
- D. **Both Standard and Enterprise**

4. Which event occurs after the WillMove event?

- A. **MoveComplete**
- B. **WillChangeField**
- C. **MoveNext**
- D. **MoveLast**

5. Which property of a data-bound control needs to be set in order to bind it to an ADO data control?

- A. **RecordSource**
- B. **ConnectionString**
- C. **DataSourceName**
- D. **DataSource**

Notes